Credit Card Approval Prediction

Project Description:

The **Credit Card Approval Prediction** project is a machine learning-based web application designed to predict whether a credit card application is likely to be approved or rejected based on an applicant's personal and financial information. The project uses the **Application Record** and **Credit Record** datasets from Kaggle for data preprocessing, feature engineering, and model training. A **Logistic Regression** algorithm is used to build the prediction model due to its effectiveness in binary classification tasks.

The application is developed using **Python**, **Flask**, **Scikit-learn**, **Pandas**, **NumPy**, **HTML**, and **CSS**. It provides a simple and user-friendly interface where users can enter applicant details and receive an instant prediction. The project is version-controlled using **GitHub** and deployed on **Render**, making it easily accessible through a web browser. This solution helps automate the credit card approval process, improving decision-making speed, accuracy, and efficiency.

Scenarios

Scenario 1:

A user applies for a credit card by entering personal details such as gender, annual income, employment status, education level, family status, and housing type through the web application. The system processes the information using the trained machine learning model and instantly predicts whether the credit card application is likely to be approved or rejected.

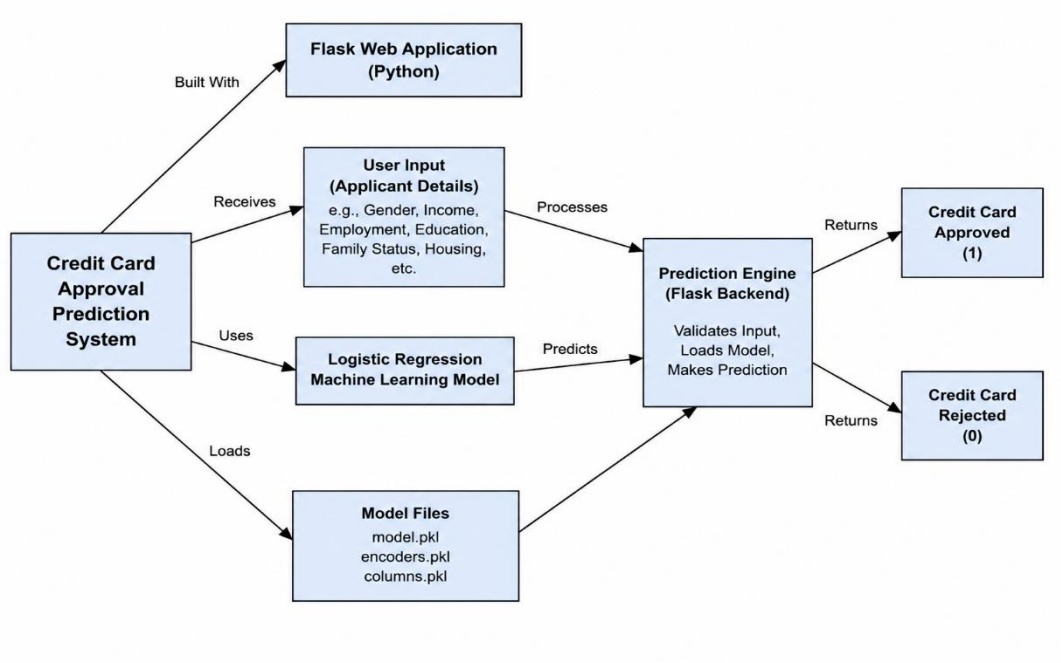
Scenario 2:

A bank employee uses the application to evaluate multiple customer applications. By entering each applicant's financial and personal information, the system quickly provides credit card approval predictions, enabling faster and more consistent decision-making while reducing manual effort.

Scenario 3:

A customer wants to check their eligibility before submitting a credit card application. After providing the required applicant details through the user-friendly interface, the system analyzes the data using the Logistic Regression model and displays the prediction result, helping the customer understand the likelihood of approval before applying.

Technical Architecture:



Description: The **Credit Card Approval Prediction System** is developed as a web-based application using the **Flask** framework in Python. Users access the application through a simple web interface and enter applicant details such as gender, annual income, employment status, education level, family status, housing type, and other required information. These inputs are received by the Flask application, which validates the data before passing it to the prediction engine for processing.

The application uses a pre-trained **Logistic Regression Machine Learning Model** to analyze the applicant's information. The prediction engine loads the required model files (model.pkl, encoders.pkl, and columns.pkl) to preprocess the input and generate a prediction. Based on the model's output, the system returns one of two possible results: **Credit Card Approved (1)** or **Credit Card Rejected (0)**. This architecture enables fast, reliable, and automated credit card approval prediction through an easy-to-use web application.

Pre-requisites:

- **Python Programming Proficiency:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Fundamentals:** Scikit-learn Documentation
- **Data Analysis & Preprocessing**: Pandas and NumPy Documentation
- **Frontend Development Skills:** W3Schools HTML & CSS Tutorials
- **Version Control with Git & GitHub:** Git Documentation
- **Development Environment Setup:** Python Virtual Environment & Flask Installation Guide

- **Cloud Deployment on Render**: Render Documentation

Project Workflow:

Milestone 1: Data Collection

• **Activity 1.1:** Download the Application Record and Credit Record datasets and prepare them for analysis and machine learning model development.

Milestone 2: Visualizing and Analysing the Data

- **Activity 2.1:** Import the required Python libraries for data analysis, visualization, and machine learning.
- **Activity 2.2:** Read and explore the application and credit datasets to understand their structure, features, and target variable.
- **Activity 2.3:** Perform univariate analysis to examine the distribution of applicant attributes such as income, education, employment, and family status.
- **Activity 2.4:** Conduct multivariate analysis to identify relationships between applicant features and credit card approval status.
- **Activity 2.5:** Perform descriptive statistical analysis to summarize key characteristics of the datasets and identify important trends.

Milestone 3: Data Pre-Processing

- **Activity 3.1**: Identify and remove duplicate records to improve data quality**.**
- **Activity 3.2**: Detect and handle missing values to ensure dataset completeness**.**
- **Activity 3.3:** Merge the application and credit datasets and perform data cleaning to create a consistent dataset.
- **Activity 3.4:** Apply feature engineering techniques to generate meaningful features for credit card approval prediction.
- **Activity 3.5:** Convert categorical variables into numerical representations using encoding techniques suitable for machine learning algorithms.

Milestone 4: Model Building

- **Activity 4.1** Train a Logistic Regression model and evaluate its credit card approval prediction performance.
- **Activity 4.2:** Train a Decision Tree model and compare its performance with other models**.**
- **Activity 4.3:** Train a Random Forest model and analyze its classification accuracy.
- **Activity 4.4:** Compare the performance of all trained models using evaluation metrics such as Accuracy, Precision, Recall, and F1-Score, and select the best-performing model for deployment.

Milestone 5: Application Building

- **Activity 5.1:** Design and develop HTML pages to provide a simple and user-friendly interface for entering applicant details.
- **Activity 5.2:** Develop the Flask web application and integrate the trained Logistic Regression model for prediction.
- **Activity 5.3:** Run, test, validate, and deploy the application on **Render** to provide accurate credit card approval predictions through a web interface.

Milestone 6: Conclusion

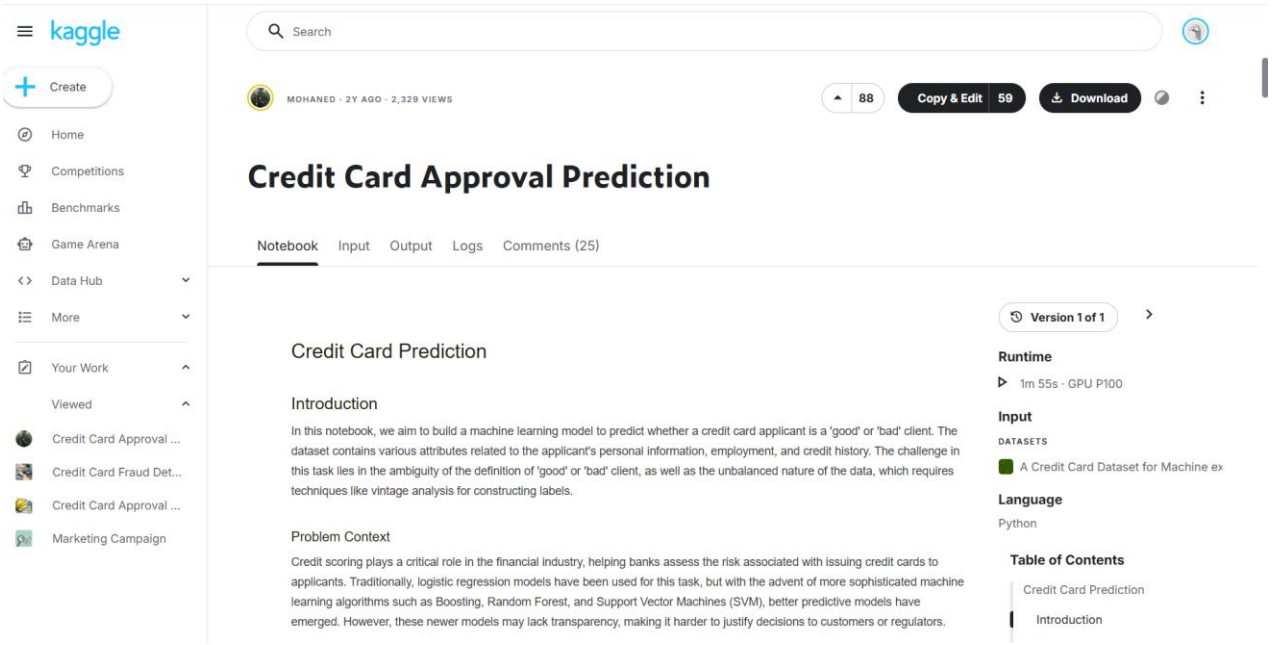
Milestone 1: Data Collection

Activity 1.1: Download the Application Record and Credit Record Datasets

In this activity, the Application Record and Credit Record datasets are downloaded from Kaggle and prepared for machine learning model development. The datasets are extracted and stored in the project directory for further preprocessing and analysis. The structure, features, and data quality of both datasets are verified before proceeding with model development.

Steps:

- Visit the Kaggle website.
- Search for the Credit Card Approval Prediction dataset.
- Download the dataset.
- Extract the ZIP file.
- Copy application_record.csv and credit_record.csv into the project folder.
- Verify that both datasets are available for analysis.



Milestone 2: Visualizing and Analysing the Data

Activity 2.1: Import the Required Python Libraries

- Import the required Python libraries such as **Pandas**, **NumPy**, **Matplotlib**, **Seaborn**, and **Scikit-learn** for data analysis, visualization, and machine learning.

train_model.py

```python
import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

```

Activity 2.2: Read and Explore the Dataset

Load the Application Record and Credit Record datasets using Pandas. Display the first few records, column names, data types, missing values, and dataset information to understand the overall structure.

```
(venv) PS C:\Users\pavan\OneDrive\Desktop\credit card> python check_columns.py
========== Application Record ==========
['ID', 'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'CNT_CHILDREN', 'AMT_INCOME_TOTAL', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE', 'NAME_F
AMILY_STATUS', 'NAME_HOUSING_TYPE', 'DAYS_BIRTH', 'DAYS_EMPLOYED', 'FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE', 'FLAG_EMAIL', 'OCCUPATION_TYP
E', 'CNT_FAM_MEMBERS']
```

```
========== Credit Record ==========
['ID', 'MONTHS_BALANCE', 'STATUS']
```

Activity 2.3: Perform Univariate Analysis

Analyze individual applicant features such as annual income, education level, occupation type, family status, and age using histograms, count plots, and bar charts.

Activity 2.4: Perform Multivariate Analysis

Descriptive statistical analysis was performed to summarize the numerical characteristics of the dataset including mean, median, standard deviation, minimum, maximum, and quartiles.

Activity 2.5: Perform Descriptive Statistical Analysis

Descriptive statistical analysis was performed to summarize the numerical characteristics of the dataset including mean, median, standard deviation, minimum, maximum, and quartiles.

Milestone 3: Data Pre-Processing

Activity 3.1: Identify and Remove Duplicate Records

Duplicate records were checked in both the Application Record and Credit Record datasets using the Pandas duplicated() function. The analysis showed that there were no duplicate records in either dataset. Therefore, no duplicate removal was required, ensuring that the data quality was maintained before preprocessing and model training.

```
(venv) PS C:\Users\pavan\OneDrive\Desktop\credit card> python check_columns.py
Application duplicates: 0
Credit duplicates: 0
```

Activity 3.2: Identify and Remove Duplicate Records

Missing values in the Application Record and Credit Record datasets were identified using the Pandas isnull().sum() function. The analysis showed that some columns, such as OCCUPATION_TYPE, contained missing values, while the Credit Record dataset had no missing values. Appropriate preprocessing techniques were applied during data preparation to handle missing values before model training.

```
(venv) PS C:\Users\pavan\OneDrive\Desktop\credit card> python check_columns.py
============================================================
APPLICATION RECORD - Missing Values
============================================================
ID                          0
CODE_GENDER                 0
FLAG_OWN_CAR                0
FLAG_OWN_REALTY             0
CNT_CHILDREN                0
AMT_INCOME_TOTAL            0
NAME_INCOME_TYPE            0
NAME_EDUCATION_TYPE         0
NAME_FAMILY_STATUS          0
NAME_HOUSING_TYPE           0
DAYS_BIRTH                  0
DAYS_EMPLOYED               0
FLAG_MOBIL                  0
FLAG_WORK_PHONE             0
FLAG_PHONE                  0
FLAG_EMAIL                  0
OCCUPATION_TYPE        134203
CNT_FAM_MEMBERS             0
dtype: int64

============================================================
CREDIT RECORD - Missing Values
============================================================
ID               0
MONTHS_BALANCE   0
STATUS           0
```

Activity 3.3: Merge the Application and Credit Datasets and Perform Data Cleaning

The Application Record and Credit Record datasets were merged using the common ID field to create a unified dataset for model development. After merging, the dataset was examined to verify its structure, columns, and overall quality. Basic data cleaning was performed by checking for missing values and ensuring the merged dataset was suitable for feature engineering and machine learning.

```
============================================================
MERGED DATASET
============================================================

First 5 Rows:
       ID CODE_GENDER FLAG_OWN_CAR FLAG_OWN_REALTY  CNT_CHILDREN  ...  FLAG_EMAIL OCCUPATION_TYPE CNT_FAM_MEMBERS MONTHS_BALANCE STATUS
0  5008804          M           Y               Y             0  ...           0            NaN             2.0              0      C
1  5008804          M           Y               Y             0  ...           0            NaN             2.0             -1      C
2  5008804          M           Y               Y             0  ...           0            NaN             2.0             -2      C
3  5008804          M           Y               Y             0  ...           0            NaN             2.0             -3      C
4  5008804          M           Y               Y             0  ...           0            NaN             2.0             -4      C

[5 rows x 20 columns]

Dataset Shape:
(777715, 20)

Column Names:
['ID', 'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'CNT_CHILDREN', 'AMT_INCOME_TOTAL', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE', 'NAME_F
AMILY_STATUS', 'NAME_HOUSING_TYPE', 'DAYS_BIRTH', 'DAYS_EMPLOYED', 'FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE', 'FLAG_EMAIL', 'OCCUPATION_TYP
E', 'CNT_FAM_MEMBERS', 'MONTHS_BALANCE', 'STATUS']

Missing Values:
ID                       0
CODE_GENDER              0
FLAG_OWN_CAR             0
FLAG_OWN_REALTY          0
CNT_CHILDREN             0
AMT_INCOME_TOTAL         0
NAME_INCOME_TYPE         0
NAME_EDUCATION_TYPE      0
NAME_FAMILY_STATUS       0
NAME_HOUSING_TYPE        0
DAYS_BIRTH               0
DAYS_EMPLOYED            0
FLAG_MOBIL               0
FLAG_WORK_PHONE          0
FLAG_PHONE               0
FLAG_EMAIL               0
OCCUPATION_TYPE     240048
CNT_FAM_MEMBERS          0
MONTHS_BALANCE           0
STATUS                   0
dtype: int64
```

Activity 3.4: Apply Feature Engineering

Feature engineering was performed to create meaningful features from the original dataset. The applicant's age was calculated from the DAYS_BIRTH column, and employment duration was converted from DAYS_EMPLOYED into years. These transformed features improve data interpretation and help the machine learning model make more accurate credit card approval predictions.

```
============================================================
FEATURE ENGINEERING
============================================================

New Features Created:
   DAYS_BIRTH  AGE  DAYS_EMPLOYED  YEARS_EMPLOYED
0       -12005   32          -4542       12.443836
1       -12005   32          -4542       12.443836
2       -21474   58          -1134        3.106849
3       -19110   52          -3051        8.358904
4       -19110   52          -3051        8.358904

Dataset Shape:
(438557, 20)
```

Activity 3.5: Apply Feature Engineering

The categorical features in the dataset were converted into numerical values using the Label Encoding technique. This preprocessing step transformed text-based attributes such as gender, income type, education level, family status, housing type, and occupation into integer values. Encoding categorical variables ensured that the dataset was compatible with machine learning algorithms and improved the efficiency of model training.

```
============================================================
ENCODED DATASET
============================================================
   CODE_GENDER  FLAG_OWN_CAR  FLAG_OWN_REALTY  NAME_INCOME_TYPE  NAME_EDUCATION_TYPE  NAME_FAMILY_STATUS  NAME_HOUSING_TYPE  OCCUPATION_TYPE
0            1             1                1                 4                    1                   0                  4                 18
1            1             1                1                 4                    1                   0                  4                 18
2            1             1                1                 4                    4                   1                  1                 16
3            0             0                1                 0                    4                   3                  1                 14
4            0             0                1                 0                    4                   3                  1                 14

Data Types After Encoding:
CODE_GENDER            int64
FLAG_OWN_CAR           int64
FLAG_OWN_REALTY        int64
NAME_INCOME_TYPE       int64
NAME_EDUCATION_TYPE    int64
NAME_FAMILY_STATUS     int64
NAME_HOUSING_TYPE      int64
OCCUPATION_TYPE        int64
dtype: object
```

Milestone 4: Model Building

Activity 4.1: Train Logistic Regression Model

A Logistic Regression model was trained using the preprocessed dataset. The model's performance was evaluated using accuracy and a classification report to measure its prediction capability.

```
Logistic Regression Accuracy: 0.9839550191991223
```

Activity 4.2: Train a Decision Tree Model

A Decision Tree classifier was trained on the processed dataset to classify credit card approval applications. The model's performance was evaluated using prediction accuracy and compared with other machine learning models.

```
Decision Tree Accuracy: 0.9810751508502469
```

Activity 4.3: Train a Random Forest Model

A Random Forest classifier consisting of multiple decision trees was trained to improve classification performance. The model was evaluated using the accuracy score obtained on the testing dataset.

```
Random Forest Accuracy: 0.982309380142622
```

Activity 4.4: Compare the Models

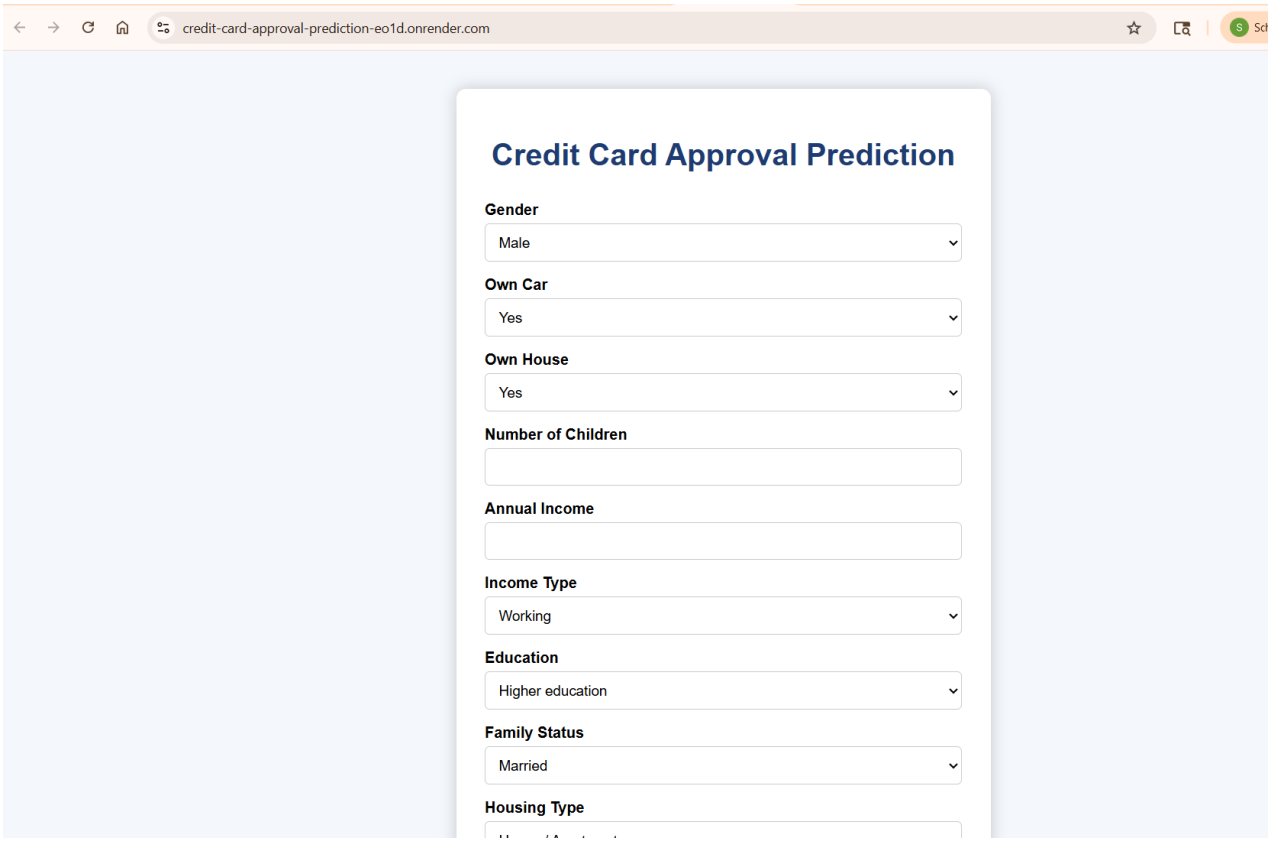
The performances of Logistic Regression, Decision Tree, and Random Forest models were compared using the accuracy metric. The model with the highest accuracy was selected as the final prediction model and saved as model.pkl for integration into the Flask web application.

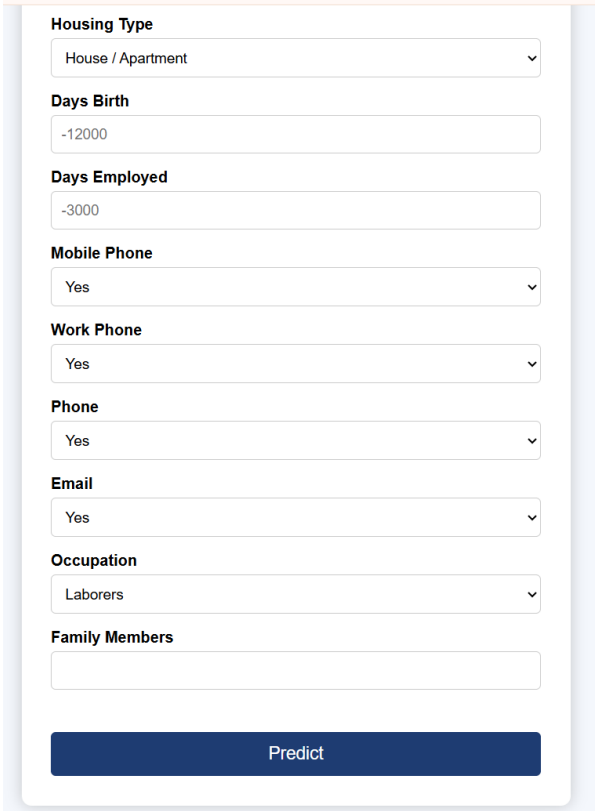
```
Logistic Regression Accuracy: 0.9839550191991223
Decision Tree Accuracy: 0.9810751508502469
Random Forest Accuracy: 0.982309380142622

Best Model: Logistic Regression

Model Saved Successfully!
```

Milestone 5: Application Building

Activity 5.1: Design and Develop HTML Pages

The user interface of the Credit Card Approval Prediction System was developed using HTML and CSS. A simple and user-friendly web page was designed to collect applicant information such as gender, income, education, employment status, family details, and housing information. The interface allows users to enter the required details and submit the application for prediction.


credit-card-approval-prediction-eo1d.onrender.com
Credit Card Approval Prediction
Gender
Male
Own Car
Yes
Own House
Yes
Number of Children
Annual Income
Income Type
Working
Education
Higher education
Family Status
Married
Housing Type


Housing Type
House / Apartment
Days Birth
-12000
Days Employed
-3000
Mobile Phone
Yes
Work Phone
Yes
Phone
Yes
Email
Yes
Occupation
Laborers
Family Members
Predict

Activity 5.2: Develop the Flask Web Application and Integrate the Trained Model

A Flask web application was developed to integrate the trained machine learning model with the user interface. The application receives applicant details from the HTML form, preprocesses the input using the saved encoders, loads the trained model (model.pkl), and predicts whether the credit card application will be approved or rejected. The prediction result is then displayed to the user through the web interface.

app.py

```python
from flask import Flask, render_template, request
import pandas as pd
import joblib

app = Flask(__name__)

# Load saved files
model = joblib.load("model.pkl")
encoders = joblib.load("encoders.pkl")
columns = joblib.load("columns.pkl")


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    try:

        data = {
            "CODE_GENDER": request.form["CODE_GENDER"],
            "FLAG_OWN_CAR": request.form["FLAG_OWN_CAR"],
            "FLAG_OWN_REALTY": request.form["FLAG_OWN_REALTY"],
            "CNT_CHILDREN": int(request.form["CNT_CHILDREN"]),
            "AMT_INCOME_TOTAL": float(request.form["AMT_INCOME_TOTAL"]),
            "NAME_INCOME_TYPE": request.form["NAME_INCOME_TYPE"],
            "NAME_EDUCATION_TYPE": request.form["NAME_EDUCATION_TYPE"],
            "NAME_FAMILY_STATUS": request.form["NAME_FAMILY_STATUS"],
            "NAME_HOUSING_TYPE": request.form["NAME_HOUSING_TYPE"],
            "DAYS_BIRTH": int(request.form["DAYS_BIRTH"]),
            "DAYS_EMPLOYED": int(request.form["DAYS_EMPLOYED"]),
            "FLAG_MOBIL": int(request.form["FLAG_MOBIL"]),
            "FLAG_WORK_PHONE": int(request.form["FLAG_WORK_PHONE"]),
            "FLAG_PHONE": int(request.form["FLAG_PHONE"]),
            "FLAG_EMAIL": int(request.form["FLAG_EMAIL"]),
            "OCCUPATION_TYPE": request.form["OCCUPATION_TYPE"],
```

```
            "FLAG_EMAIL": int(request.form["FLAG_EMAIL"]),
            "OCCUPATION_TYPE": request.form["OCCUPATION_TYPE"],
            "CNT_FAM_MEMBERS": float(request.form["CNT_FAM_MEMBERS"])
        }

        df = pd.DataFrame([data])

        # Encode categorical columns
        for col in encoders:
            value = str(df.loc[0, col])

            if value not in encoders[col].classes_:
                value = encoders[col].classes_[0]

            df[col] = encoders[col].transform([value])

        # Arrange columns
        df = df[columns]

        prediction = model.predict(df)[0]

        if prediction == 0:
            result = "Credit Card Approved"
        else:
            result = "Credit Card Rejected"

        return render_template("result.html", prediction=result)

    except Exception as e:
        return str(e)


if __name__ == "__main__":
    app.run(debug=True)
```

Activity 5.3: Run, Test, Validate, and Deploy the Application

The Flask application was tested locally to verify prediction accuracy and application functionality. After successful validation, the project was deployed on the Render cloud platform, allowing users to access the application through a public URL. The deployed application successfully predicts credit card approval based on applicant information entered through the web interface.

```
(venv) PS C:\Users\pavan\OneDrive\Desktop\credit card> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 123-585-036
 * Detected change in 'C:\\Users\\pavan\\OneDrive\\Desktop\\credit card\\check_columns.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 123-585-036
```

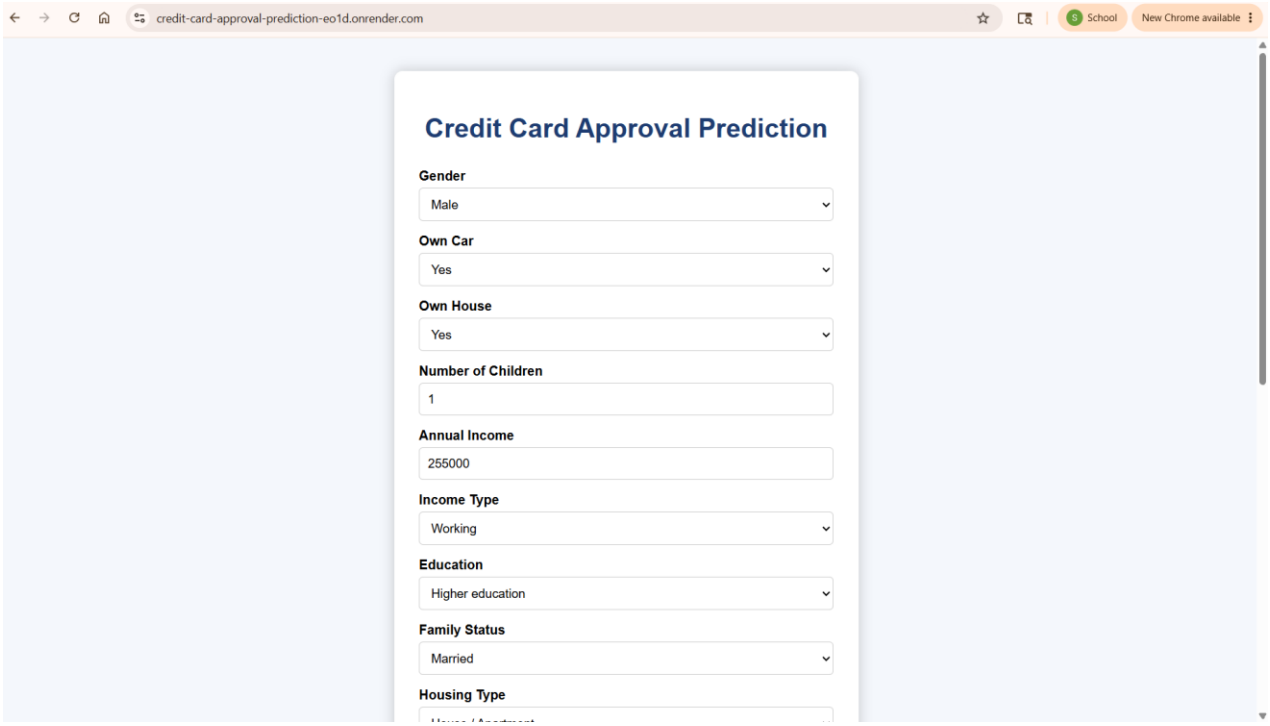
All logs | Search logs | Jun 30, 10:35 AM - 10:40 AM | GMT+5:30

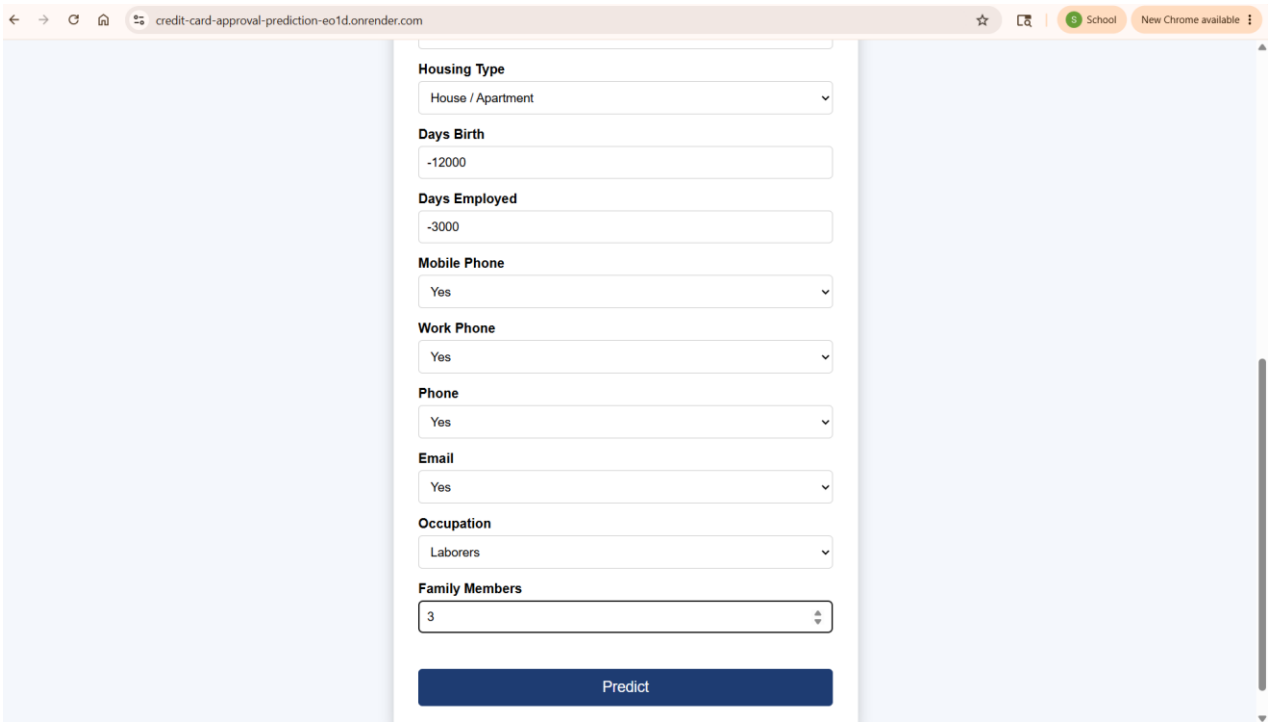
Jun 30

```
10:38:10 AM  [jcqpr]  ==> Running 'gunicorn app:app'
10:38:42 AM           ==> No open ports detected, continuing to scan...
10:38:42 AM           ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
10:39:00 AM  [jcqpr]  [2026-06-30 05:09:00 +0000] [59] [INFO] Starting gunicorn 23.0.0
10:39:00 AM  [jcqpr]  [2026-06-30 05:09:00 +0000] [59] [INFO] Listening at: http://0.0.0.0:10000 (59)
10:39:00 AM  [jcqpr]  [2026-06-30 05:09:00 +0000] [59] [INFO] Using worker: sync
10:39:00 AM  [jcqpr]  [2026-06-30 05:09:00 +0000] [74] [INFO] Booting worker with pid: 74
10:39:01 AM  [jcqpr]  127.0.0.1 - - [30/Jun/2026:05:09:01 +0000] "HEAD / HTTP/1.1" 200 0 "-" "Go-http-client/1.1"
10:39:08 AM           ==> Your service is live 🎉
10:39:08 AM  [jcqpr]  127.0.0.1 - - [30/Jun/2026:05:09:08 +0000] "GET / HTTP/1.1" 200 5108 "-" "Go-http-client/2.0"
10:39:08 AM           ==>
10:39:08 AM           ==> ///////////////////////////////////////////////////////////
10:39:08 AM           ==>
10:39:08 AM           ==> Available at your primary URL https://credit-card-approval-prediction-eo1d.onrender.com
10:39:08 AM           ==>
10:39:08 AM           ==> ///////////////////////////////////////////////////////////
```

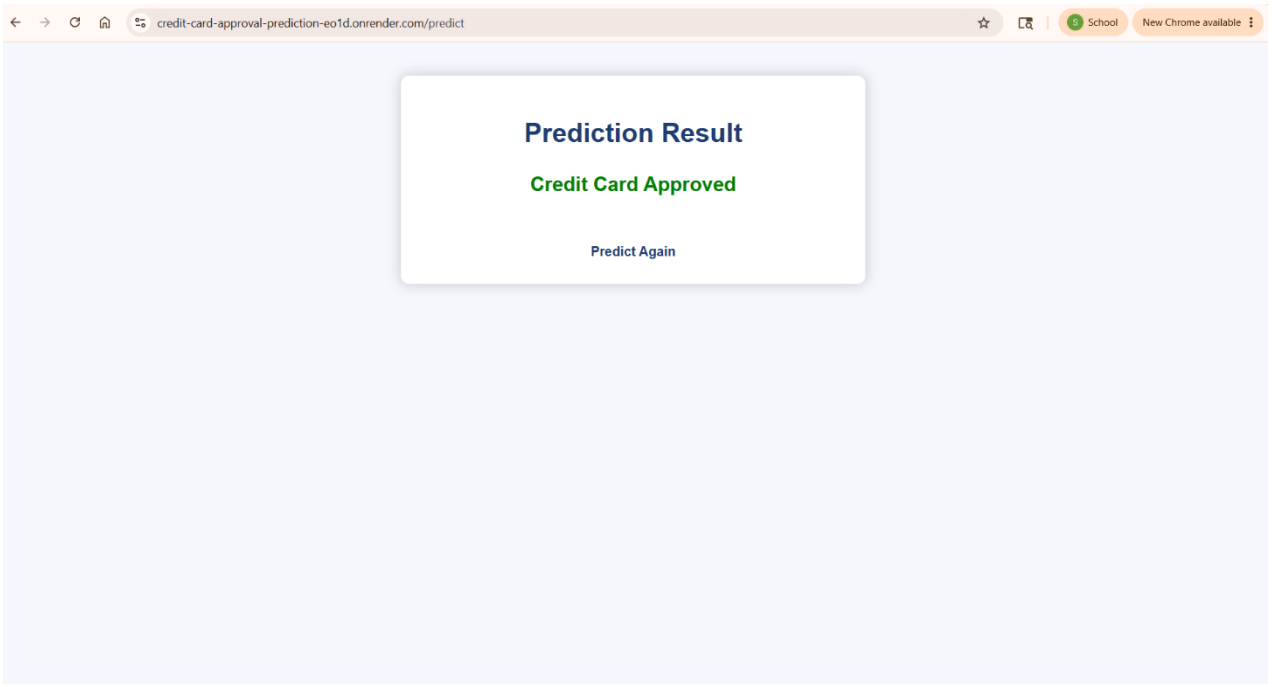
Need better ways to work with logs? Try the Render CLI, Render MCP Server, or set up a log stream integration

Input Section:





Results Section:



Description: After the user submits the application details, the system processes the input using the trained machine learning model and displays the prediction result. The result indicates whether the credit card application is Approved or Rejected, enabling users to receive an instant decision.

Conclusion:

The Credit Card Approval Prediction System successfully demonstrates how machine learning can be utilized to automate and enhance the credit card approval process in the banking and financial sector. The project analyzes important applicant details such as income type, employment status, family status, housing type, and credit history to determine whether a credit card application should be approved or rejected.

The system follows a complete machine learning pipeline, including data collection, data preprocessing, data visualization, feature engineering, categorical encoding, model training, testing, and performance evaluation. Various classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost are implemented and compared to identify the most accurate and efficient prediction model.

The best-performing model is integrated into a Flask-based web application that provides a user-friendly interface for bank staff or users to enter applicant details and instantly receive approval prediction results. Additionally, the project incorporates IBM Watson Machine Learning for cloud deployment, enabling scalable and real-time prediction services.

Overall, the project provides hands-on experience in machine learning, data analysis, Flask web development, cloud deployment, and financial risk assessment while addressing a real-world banking and credit approval challenge efficiently and accurately.